

Lab 2 recap — Explaining a Model with SHAP

OPIM 5512 · Module 2 (Explainable AI) · Week 4. A short recap of what we covered, how to **finish** if you ran out of time, and how to **submit**. Keep this next to the [instructions](#) and the [20-step sheet](#).

The one idea to remember

SHAP is a receipt, not a ranking. For a *single* prediction it gives you an itemized, **signed** list of how many megawatts each feature added or subtracted — and the line items **add up exactly** to the prediction:

base value (average prediction)	14,952 MW	
hour_of_day = 17 (5 PM)	+3,000	
dewpoint_f = 71 (humid)	+2,300	
temp_f = 85 (hot)	+2,200	
weekend = 0 (weekday)	+900	
wind_kt = 12 (some cooling)	-270	
this hour's prediction	23,082 MW	← adds up exactly

That "it sums to the answer" guarantee is the thing permutation importance and partial dependence **can't** give you.

Three tools, three boss-questions — use all three

Tool	Answers	Great for	Blind spot
Permutation importance	"What matters?"	a fast, model-agnostic ranking	global only; correlated features can fool it
Partial dependence	"How does it behave?"	the shape (up / down / curvy)	it's an average — hides who's different
SHAP	"Why this one?"	per-row, signed, adds up — and rolls up to a global view for free	costs more compute

PI ranks · PDP shapes · SHAP itemizes. Only SHAP explains a single decision — and hands you the global view on the way. In practice you show all three, and people get excited about your model.

What we built in the lab

- Partner A — global:** `shap.plots.beeswarm(shap_values)` → each dot is one hour; color = the feature's value; left/right = its push in MW. Read top-to-bottom, **hour of day, dewpoint, temp** do the work.
- Partner B — local:** find the peak-demand hour, then `shap.plots.waterfall(shap_values[i])` → the receipt above, for that one hour.
- The report writes itself.** Drop your PNGs into `images/` with the **exact** given names and `REPORT.md` auto-populates through soft-referencing — no editing needed. Merge both partners' PRs and all four plots appear.

Finish it (if you ran out of time)

1. **Partner A** open `notebooks/Lab2_A_Global_SHAP.ipynb`, **Partner B** open `notebooks/Lab2_B_Local_SHAP.ipynb` — from **your** repo, on **your** `dev-` branch.
 2. **Runtime** → **Run all**, write your **one SHAP line**, run the download cell.
 3. Drag the PNGs into `images/` (keep the exact names — no `(1)`), **Commit** → **Push** on your `dev-` branch.
 4. **Compare & pull request** → **add your partner as Reviewer** → approve each other → **Merge** both.
 5. **Submit**: screenshot **Insights** → **Network** (the loop going both ways) and turn it in.
-

The three things that tripped people up (and the fix)

- **Committed or dragged files while on `main`**. Happens to everyone — it happened live in class. Fix: **Current branch** → **New branch** `dev-...` → GitHub Desktop asks "**Bring my changes to the new branch**" → yes. Work moves over, `main` stays clean, nothing lost.
 - **Forgot to add a Reviewer**. No reviewer = no approval = no loop = no credit. Add your partner on **every** PR.
 - **Renamed a plot / left a `(1)` on it**. The report references exact filenames — keep them exactly as given.
-

Where SHAP fits from here

From now on, whenever you fit a model, run SHAP on it: *what's driving it?* and, when it makes a bad miss, *why?* Real example from class — an hourly-retrained random forest + SHAP on a football-field-sized paper machine with ~1,000 sensors surfaced three suspicious sensors trending with a KPI failure; one was the culprit. SHAP is a **forensic tool**. (Working with images? SHAP highlights contributing pixels for tree/flattened models; use **Grad-CAM** for CNNs.)

Housekeeping

- **Solutions** are posted so you can check your work.
- **Next class is in person in two weeks** — let's all be in the room; it's more fun.
- **Office hours**: Friday 1:30 (Teams). Email me twice if you're stuck — I'd rather hear from you.